

# Claims Liability Predictive Model

## Step 1: Environment Setup & Data Loading

```
In [4]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import TweedieRegressor
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
```

```
In [5]: # Load datasets
freq = pd.read_csv('data/freMTPL2freq.csv')
sev = pd.read_csv('data/freMTPL2sev.csv')

# Initial check for data integrity
print(f"Frequency Data: {freq.shape}")
print(f"Severity Data: {sev.shape}")
```

Frequency Data: (678013, 12)

Severity Data: (26639, 2)

## Step 2: Data Integration (Merging & Cleaning)

In insurance analytics, you must link claims to their specific policies. Since one policy can have multiple claims, we aggregate the severity data first.

1. Aggregate Severity: Group sev by IDpol to get the total ClaimAmount per policy.
2. Merge: Use a "left join" to attach these amounts to the freq table.
3. Fill Nulls: Policies with no claims will have NaN values; fill these with 0.
4. Handle Exposure: Ensure Exposure is clipped at 1.0 (some policies represent more than a year, but for modeling, we cap it).

```
In [6]: # 1. Aggregate claim amounts by policy ID
sev_agg = sev.groupby('IDpol').agg({'ClaimAmount': 'sum'}).reset_index()

# 2. Merge with frequency data
df = pd.merge(freq, sev_agg, on='IDpol', how='left')

# 3. Fill non-claims with 0
df['ClaimAmount'] = df['ClaimAmount'].fillna(0)

# 4. Data Quality: Cap exposure at 1 and drop zero-exposure rows
df['Exposure'] = df['Exposure'].clip(upper=1)
df = df[df['Exposure'] > 0]
```

## Step 3: Feature Engineering & Pre-processing

You must transform raw columns like Area and VehBrand into a format a machine learning model can understand.

- Define Features: Separate your numerical features (e.g., DrivAge, VehAge, Density) from categorical ones (e.g., Area, VehBrand, VehGas).
- Target Variable: We are predicting Pure Premium, which is ClaimAmount / Exposure.

```
In [7]: # Categorical and Numerical identification
cat_features = ['Area', 'VehBrand', 'VehGas', 'Region']
num_features = ['VehPower', 'VehAge', 'DrivAge', 'BonusMalus', 'Den

# Create the target variable
df['PurePremium'] = df['ClaimAmount'] / df['Exposure']

# Split data into Train and Test sets
df_train, df_test = train_test_split(df, test_size=0.2, random_stat
```

## Step 4: Building the "Tweedie" Model Pipeline

Insurance claims data is "zero-inflated" (most people don't crash). The Tweedie Regressor is the industry standard for this type of distribution.

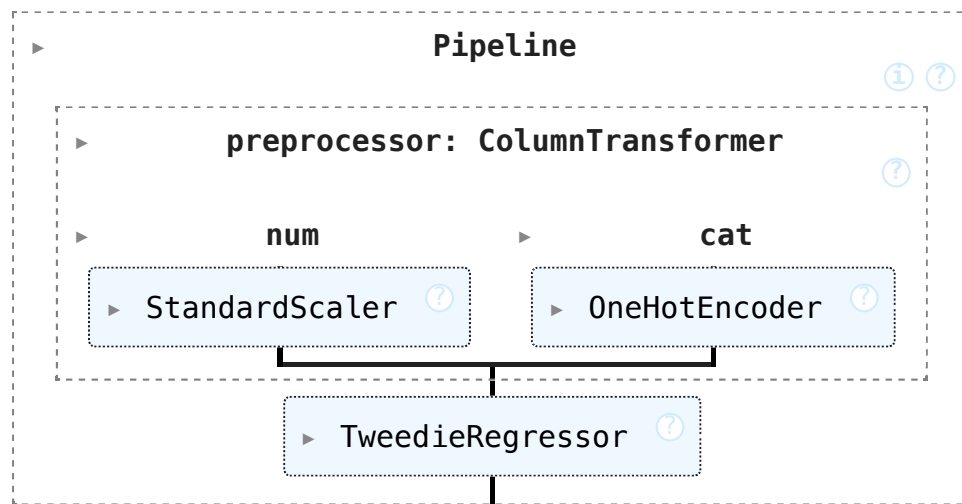
```
In [8]: # Define the pre-processing steps
preprocessor = ColumnTransformer(
    transformers=[
        ('num', StandardScaler(), num_features),
        ('cat', OneHotEncoder(handle_unknown='ignore'), cat_features)
    ])

# Define the Tweedie Model (power=1.5 is common for Pure Premium)
model = TweedieRegressor(power=1.5, link='log', max_iter=300)

# Create the full pipeline
clf = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('regressor', model)
])

# Fit the model using 'Exposure' as a sample weight
clf.fit(df_train[cat_features + num_features],
        df_train['PurePremium'],
        regressor__sample_weight=df_train['Exposure'])
```

Out[8]:



## Step 5: Evaluation & Performance Narrative

To stand out, evaluate your model using metrics that prove "Business Value".

- **Mean Absolute Error (MAE):** Tells you how much your prediction deviates on average.
- **Gini Coefficient:** Shows how well the model separates "high-risk" from "low-risk" drivers.

```
In [9]: from sklearn.metrics import mean_absolute_error, mean_tweedie_deviance
y_pred = clf.predict(df_test[cat_features + num_features])

print(f"Mean Absolute Error: {mean_absolute_error(df_test['PurePremium'], y_pred)}")
print(f"Mean Tweedie Deviance: {mean_tweedie_deviance(df_test['PurePremium'], y_pred)}
```

```
Mean Absolute Error: 581.5228461071929
Mean Tweedie Deviance: 155.64090495595494
```

In [ ]: